

@orchestra-graph

Appears in: [DL-02](#)

orchestra/graph.py:23-28, 99-143

"...feeds their findings to a central orchestrator that applies a conflict-precedence table before anything executes (@orchestra-graph)." ... "the Orchestra precedence table blocks the purchase order while a duplicate invoice is active (@orchestra-graph)."

@orchestra-graph

orchestra/graph.py:23-28

```
23 # — Conflict precedence (code-level safety) —————
24 PRECEDENCE = {
25     ("duplicate_invoice", "stockout_risk"): "duplicate_invoice",
26     ("price_spike", "stockout_risk"): "price_spike",
27     ("supplier_unavailable", "stockout_risk"): "stockout_risk",
28 }
```

...

orchestra/graph.py:99-143

```
99 def orchestrate_node(state: OrchestraState) -> dict:
100     """Resolve conflicts between agent findings and produce decisions."""
101     llm = get_llm()
102     findings = state["all_findings"]
103     brief = state["brief"]
104
105     # Group findings by sku_id
106     by_sku: dict = {}
107     for f in findings:
108         by_sku.setdefault(f.get("sku_id"), []).append(f)
109
110     conflicts = []
111     decisions = []
112
113     for sku_id, sku_findings in by_sku.items():
114         action_types = [f["type"] for f in sku_findings]
115
116         # Detect conflict
117         conflict_pair = None
118         for a_type in action_types:
119             for b_type in action_types:
120                 if a_type != b_type:
121                     key = tuple(sorted([a_type, b_type]))
122                     if key in {tuple(sorted(k)) for k in PRECEDENCE}:
123                         conflict_pair = (a_type, b_type)
124                         break
125
126         if conflict_pair:
127             winner_type = _resolve_conflict(conflict_pair, sku_findings, llm)
128             a_f = next(f for f in sku_findings if f["type"] == conflict_pair[0])
129             b_f = next(f for f in sku_findings if f["type"] == conflict_pair[1])
130             conflicts.append({
131                 "sku_id": sku_id,
132                 "conflict": conflict_pair,
133                 "winner": winner_type,
134                 "findings": [a_f, b_f],
135             })
136             winning_findings = [f for f in sku_findings if f["type"] == winner_type]
137         else:
138             winning_findings = sku_findings
139
140         for f in winning_findings:
141             decisions.append(_finding_to_decision(f, brief))
142
143     return {"conflicts": conflicts, "decisions": decisions}
```

What it shows: The hardcoded PRECEDENCE dict — duplicate-invoice beats stockout, price-spike beats stockout — applied in `orchestrate_node` *before* execute, so a duplicate invoice blocks the related PO by rule rather than by model judgement.

Source: `orchestra/graph.py:23-28, 99-143` · image rendered by `build_snippets.py`